

CHAKRA

Developer Handoff Kit for Mayur

House of DK · Aux Services LLC · Austin, Texas

DK (USA) · Mayur (India) · Confidential

Ver 1 · May 26, 2026 · CONFIDENTIAL

Jai Shri Krishna 🙏

Dedicated to Lord Krishna by Dheeraj Kohli (DK)

1. Context — What This Kit Is

This document is the complete technical handoff from DK (Dheeraj Kohli, Austin Texas USA) to Mayur (India). It covers everything Mayur needs to convert the Chakra™ V31 HTML prototype into a live, hosted, multi-user web application.

This is not a one-time engagement. DK and Mayur will work together on an ongoing basis. DK will continue to iterate on the HTML prototype with Claude (AI assistant), and send Mayur updated versions regularly. This document defines that ongoing working process.

DK's role	Product owner, designer, philosopher. Iterates on the HTML prototype with Claude. Sends Mayur new versions with change notes.
Mayur's role	Developer. Converts HTML to live app. Applies DK's changes to the live system. Manages hosting, database, and deployment.
Claude's role	DK's AI assistant. Generates new HTML versions, writes code samples, produces this document and all supporting files.
Communication	DK is in USA. Mayur is in India. All file transfers via Google Drive. Changes communicated via WhatsApp or email with version notes.

2. System Architecture — Six Layers

Every House of DK product — now and in future — follows this exact six-layer architecture. Nothing deviates from this standard.

Layer	Technology	Purpose
Domain	GoDaddy	Holds all domain names. DNS points to Render. DK owns two domains: picturize.com and auxservicesllc.com. Everything else runs on Render subdomains (appname.onrender.com) until a custom domain is purchased.
Developer	GitHub	All code — apps and agents — in private repositories under the Aux account. Every push auto-deploys to Render.
Hosting	Render	Runs all Node.js apps: Chakra™, Picturizze, future apps. Also runs agent background services. Free tier for prototyping.
Data	PostgreSQL + Cloudflare R2	PostgreSQL for all structured data (tasks, users, metadata). Cloudflare R2 for all files and photos. Zero egress fees on R2.
Agents	LangChain.js + TypeScript	Autonomous background services: Email Agent, Task Agent, Research Agent. Scheduled or event-triggered. Hosted on Render.
AI Gateway	LiteLLM	Single interface between all apps/agents and all AI providers. Switch Anthropic to OpenAI or Gemini with one config line.

2.1 GoDaddy → Render DNS Connection

For each GoDaddy domain that points to a Render app, Mayur sets this DNS record in GoDaddy:

Record type	CNAME
Name / Host	www (or @ for root domain)
Value / Points to	appname.onrender.com
TTL	3600 (1 hour)

✓ *GoDaddy holds the domain name only. Render hosts the running code. They are connected by one DNS record. No data goes through GoDaddy.*

2.2 Current Domains

picturize.com	Points to Picturizze app on Render
picturizephotography.com	Alias — redirects to picturize.com
auxservicesllc.com	Points to Aux corporate / Chakra™ landing on Render
chakra.onrender.com	Chakra™ app — Render subdomain during prototype phase
All other apps	appname.onrender.com — Render subdomains until custom domains are purchased

⚠️ The .onrender.com suffix is permanent on Render's free tier unless a custom domain is attached. Keep app names short — chakra.onrender.com, not aux-chakra-app.onrender.com.

3. The Recommended Stack — One Standard

Every House of DK product uses this stack. Do not deviate. Standardisation is the goal — so that Dhruv, Disha, or any future team member can pick up any product and immediately understand the structure.

Code storage	GitHub — free, unlimited private repos
Hosting platform	Render — free tier for prototypes, flat predictable pricing at scale
Primary database	PostgreSQL — managed by Render, from \$7/month beyond free tier
File & image storage	Cloudflare R2 — zero egress fees, free up to 10GB
Domain registrar	GoDaddy — DK already has this. Do not move domains.
Agent framework	LangChain.js — TypeScript, runs in Node.js, same language as all apps
AI gateway	LiteLLM — one interface for Anthropic, OpenAI, Gemini and 100+ others
Authentication	bcrypt + express-session — hashed passwords, PostgreSQL session store
Backend language	Node.js + Express — all apps and all agents
Estimated monthly cost	~\$10–15/month total when fully running. No surprise bills.

4. Account Naming Convention — Aux

All platform accounts use the prefix Aux. Three characters. Lowercase. No suffixes like Services or LLC — those words are intentionally excluded from all login IDs and handles.

The Aux identity is the infrastructure layer. House of DK is the philosophical and creative direction. Individual products (Chakra™, Picturizze) are the consumer-facing brands. This separation is intentional.

This infrastructure is designed to grow. DK's son Dhruv (realtor) and daughter Disha (dietitian) may build their own products on top of this same stack. They can use the Aux infrastructure or build their own brand on top of it. The platform is family-transferable by design.

Platform	Login / Handle	URL	Purpose
GitHub	aux-dk	github.com	All code repositories
Render	aux	render.com	Hosting all apps and agents
Anthropic Console	aux-dev	console.anthropic.com	API keys for Claude / LiteLLM
Cloudflare	aux-storage	cloudflare.com	R2 storage for images and files
LiteLLM	aux-gateway	Deployed on Render	AI gateway service
Google Drive	aux-drive	drive.google.com	File transfers DK → Mayur

⚠ Store all credentials in a secure password manager. Never commit API keys to GitHub. Use .env files locally. Add .env to .gitignore always.

5. Brand Colors — What to Use and What to Avoid

Use these exact hex codes for all UI work. No substitutions.

5.1 Use These Colors

Hex Code	Name	Use for
#C9A84C	Peacock Gold	Primary brand color. Logo, wordmark, headings, CTAs, all primary brand elements.
#1A6B5A	Peacock Teal	Secondary brand color. Structural elements, dividers, category headers.
#0F4A3A	Deep Forest	Dark text on light backgrounds. Section headers, body text on gold surfaces.
#FFFFFF	White	Primary background. All main surfaces. The cleared mind.
#F5F5F5	Pearl	Secondary background. Cards, input areas, secondary panels.
#2E7D52	Emerald Green	Completion states, positive signals, progress bars.
#D4AF37	Bright Gold	Accent. Borders, dividers, progress indicators, call-to-action borders.
#00BFA5	Working Zone Teal	Double-border working state indicator in Chakra™ dashboard only.

5.2 Never Use These Colors

Hex Code	Name	Why banned
#000000	Pure Black	Banned as any primary background or dominant surface. Never use as background.
#1B3A6B	Dark Navy	Banned as dominant surface. May appear as a thin accent line only, never as a primary color.
#4A2D8C	Violet / Purple	Banned entirely from Chakra™ and all Aux products. Reserved for a separate House of DK product.

6. Shared Auth Service — Single Login Across All Apps

All House of DK apps share one authentication service. A user logs in once and is recognised across Chakra™, Picturizze, and every future app.

6.1 PostgreSQL Users Table

```
CREATE TABLE users (  
  id          SERIAL PRIMARY KEY,  
  username    VARCHAR(80) UNIQUE NOT NULL,  
  password_hash TEXT NOT NULL,  
  role        VARCHAR(20) DEFAULT 'tester',  
  created_at  TIMESTAMP DEFAULT NOW()  
);
```

6.2 User Registration — bcrypt Hashed Passwords

```
const bcrypt = require('bcrypt');  
const SALT_ROUNDS = 12;  
  
async function registerUser(db, username, password, role = 'tester') {  
  const hash = await bcrypt.hash(password, SALT_ROUNDS);  
  await db.query(  
    'INSERT INTO users (username, password_hash, role) VALUES ($1, $2, $3)',  
    [username, hash, role]  
  );  
}
```

6.3 Login + Session

```
async function loginUser(db, username, password) {  
  const res = await db.query(  
    'SELECT * FROM users WHERE username = $1', [username]  
  );  
  if (!res.rows.length) throw new Error('User not found');  
  const valid = await bcrypt.compare(password, res.rows[0].password_hash);  
  if (!valid) throw new Error('Invalid password');  
  return res.rows[0];  
}
```

6.4 Session Middleware — Stored in PostgreSQL

```
const session = require('express-session');  
const pgSession = require('connect-pg-simple')(session);
```



```
module.exports = session({
  store: new pgSession({ pool }),
  secret: process.env.SESSION_SECRET,
  resave: false,
  saveUninitialized: false,
  cookie: { maxAge: 7 * 24 * 60 * 60 * 1000, secure: true, httpOnly: true }
});
```

6.5 Auth Guard Middleware

```
function requireAuth(req, res, next) {
  if (!req.session.userId) return res.redirect('/login');
  next();
}
```

Initial user accounts to create — four accounts for prototype testing:

dk	DK — primary user, role: admin
tester1	DK's sister — regression tester, role: tester
tester2	Regression tester 2, role: tester
tester3	Regression tester 3, role: tester

⚠️ *Passwords are never stored in plain text. bcrypt with 12 salt rounds is mandatory. Never log passwords anywhere.*

7. Data Storage — Moving Off localStorage

7.1 The V31 Problem

- All 111 tasks are hardcoded in the HTML file in ITEMS_DEFAULT
- Data saved to browser localStorage — device-specific, not server-side
- Opening on a different device shows zero tasks
- Clearing browser cache wipes all data permanently
- Different users cannot have separate task lists

7.2 PostgreSQL Tasks Table

```
CREATE TABLE tasks (  
  id SERIAL PRIMARY KEY,  
  user_id INTEGER REFERENCES users(id),  
  title TEXT NOT NULL,  
  bucket VARCHAR(30) DEFAULT 'Karya',  
  weightage VARCHAR(5),  
  time_horizon VARCHAR(30),  
  time_horizon_type VARCHAR(20),  
  life_area VARCHAR(50),  
  chapter INTEGER,  
  multitask BOOLEAN,  
  completed BOOLEAN DEFAULT false,  
  entry_timestamp TIMESTAMP DEFAULT NOW(),  
  aging_days INTEGER DEFAULT 0  
);
```

7.3 API Endpoints to Build

GET /api/tasks	Fetch all tasks for the logged-in user from PostgreSQL
POST /api/tasks	Add a new task linked to user_id
PATCH /api/tasks/:id	Update a task — bucket, horizon, completion status
DELETE /api/tasks/:id	Delete a task
POST /api/claude	Proxy all Claude/LiteLLM API calls. API key stays server-side only.

8. Target Project Structure

After conversion from V31 HTML, the project must follow this exact folder structure:

```
/chakra-app
/public
  index.html      <- login page
  dashboard.html  <- Karma Kshetra™ dashboard
/js
  app.js          <- all frontend logic (extracted from HTML)
  data.js         <- GITA_CHAPTERS array only (tasks move to DB)
/css
  styles.css      <- all CSS extracted from HTML
/server
  index.js        <- Express server entry point
  auth.js         <- login / logout / session
  db.js           <- PostgreSQL connection pool
/routes
  tasks.js        <- task CRUD API
  claude.js       <- Claude/LiteLLM API proxy
.env              <- secrets (NEVER commit this file)
.gitignore        <- must include .env and node_modules
package.json
README.md
```

⚠ The `ITEMS_DEFAULT` array (111 hardcoded tasks) moves to PostgreSQL on first run. The HTML file carries zero task data after conversion.

9. Converting V31 HTML — Is It Automated?

Partially automated. Not one-go. Roughly 80% is scaffolded by Claude Code. 20% requires Mayur's judgment.

Claude Code handles (~80%)	Project folder scaffolding, extracting CSS and JS into separate files, creating server.js, package.json, .env template, .gitignore, wiring API routes
Mayur handles manually (~20%)	Reviewing generated code for accuracy, confirming database schema, setting environment variables on Render, deploying and testing

9.1 Main Conversion Prompt for Claude Code

Upload KarmaKshetra_DK_051626_v31.html first. Then paste this prompt:

```
You are converting a single-file HTML prototype called Karma Kshetra™
(Chakra™ system) into a structured Node.js project.
File: KarmaKshetra_DK_051626_v31.html

Do the following in order:
1. Create project structure: /public, /js, /css, /server,
   package.json, .env.template, .gitignore
2. Extract all CSS into css/styles.css
3. Extract all JS into js/app.js and js/data.js
   (data.js = GITA_CHAPTERS only. ITEMS_DEFAULT moves to DB.)
4. Create server/index.js with Node.js + Express
5. Move ALL Anthropic/LiteLLM API calls into server/routes/claude.js
   as POST /api/claude. API key via process.env only. Never in browser.
6. Add PostgreSQL pool in server/db.js using DATABASE_URL from .env
7. Replace all localStorage with GET/POST/PATCH /api/tasks calls
8. Add bcrypt auth with express-session stored in PostgreSQL
9. Each task row must include user_id – users see only their own tasks
10. Verify Back to Basics PDF generation still works (client-side only)
11. Keep app names short for Render: chakra.onrender.com

Do NOT change any UI, color, logic, or brand terminology.
All trademark names must remain exactly as written.
No 'Created by Claude' anywhere in any file or UI.
```

9.2 Three Manual Steps Mayur Must Do

1. Create Anthropic account at console.anthropic.com and generate an API key under the aux-dev login

2. Create Render account (aux login), connect the GitHub repository, create a PostgreSQL database
3. Add all environment variables in Render's dashboard under the Environment tab:

```
ANTHROPIC_API_KEY = sk-ant-...  
DATABASE_URL      = postgresql://... (from Render PostgreSQL panel)  
SESSION_SECRET   = a-long-random-string-minimum-32-characters  
LITELLM_URL       = https://aux-gateway.onrender.com (when ready)
```

9.3 LiteLLM Gateway Setup Prompt

After the main conversion is live, paste this second prompt to add the AI gateway:

```
Add LiteLLM as an AI gateway between all app API calls and LLM providers.  
1. Create a new /litellm service on Render  
2. Create litellm/config.yaml with Anthropic as primary,  
   OpenAI as fallback, Gemini as secondary fallback  
3. Update server/routes/claude.js to call LiteLLM proxy URL  
   instead of calling Anthropic directly  
4. Store all provider API keys in Render environment variables  
5. Add LITELLM_URL to .env
```

10. Ongoing Change Process — DK to Mayur

This is the most important section for the long-term working relationship. DK will continue to iterate on the HTML prototype with Claude. This is the agreed process for how changes flow from DK in the USA to Mayur in India.

10.1 The Workflow

Step	Who	What
1	DK + Claude (USA)	DK describes the change needed. Claude generates an updated HTML file. The file is versioned: KarmaKshetra_DK_MMDDYY_vNN.html
2	DK (USA)	DK reviews the HTML in the browser. Confirms it looks and works correctly.
3	DK (USA)	DK uploads the new HTML file to Google Drive (Aux Drive folder: House of DK / Chakra / Versions). DK sends Mayur a WhatsApp or email with two lines: the new filename and what changed.
4	Mayur (India)	Mayur downloads the new HTML from Google Drive. Reads the change note. Identifies which parts of the live project are affected.
5	Mayur (India)	Mayur applies the changes to the live project on Render. This is not a full reconversion — only the changed sections are updated.
6	Mayur (India)	Mayur confirms the live URL is working and sends DK a WhatsApp confirmation with the live URL to verify.
7	DK (USA)	DK opens the live URL and confirms. If satisfied, the version is marked stable.

10.2 How Much Work Is Each Update for Mayur?

This depends on what changed in the HTML. Most updates are small.

Type of Change	Mayur's Effort	Examples
UI only — colors, labels, layout, text	Under 30 minutes	Rename a tab, change a color, adjust tile size
New input field or display element	30–60 minutes	Add a new chip option, add a tile to Gather

Logic change — scoring, calculation	1–2 hours	Change how color buffer is calculated
New data field added to tasks	2–3 hours	New column in PostgreSQL + API update
Completely new tab or major section	Half day	New tab with its own data and logic

10.3 Change Note Format

When DK sends Mayur a new HTML version, the change note must follow this simple format. Two lines minimum:

```
File: KarmaKshetra_DK_052626_v32.html
Changes: Added Q4 time horizon option. Fixed overdue star size.
         Renamed Kshetra View third tile label.
Priority: Normal [or: Urgent / Low]
```

✓ *A precise two-line change note cuts Mayur's effort in half. He knows exactly where to look without reading the entire HTML file.*

10.4 Version Naming Convention

HTML prototype files	KarmaKshetra_DK_MMDDYY_vNN.html
Documents	DocName_MMDDYY.docx
Current prototype	KarmaKshetra_DK_051626_v31.html
Next version example	KarmaKshetra_DK_052626_v32.html

⚠ *Version numbers are sequential and never reset. V31 is always V31. The next version is always V32 regardless of what date it is created.*

10.5 Google Drive Folder Structure

All files between DK and Mayur flow through this Google Drive structure:

```
House of DK /
  Chakra /
    Versions /      <- all HTML prototype files from DK
    Mayur Kit /     <- this document and all support files
    Live Builds /   <- Mayur's deployed build notes
    Brand /         <- brand guidelines, color system
```

11. Go-Live and Production Releases

Mayur does not decide when anything goes live. All production releases — prototype, beta, public — are confirmed by DK before deployment.

► *Confirm all go-live dates and production deployments with DK before executing. No exceptions. This applies to the prototype launch, beta release, and any public-facing deployment.*

Prototype go-live	Confirm with DK. Internal testing only — DK + 3 testers.
Beta release	Confirm with DK. Wider audience, not public.
Public launch	Confirm with DK. Requires custom domain to be live.
Any domain changes	Confirm with DK before touching GoDaddy DNS settings.
Database schema changes	Confirm with DK if they affect existing user data.

12. Files Attached — Send All to Mayur via Google Drive

Upload all files below to: House of DK / Chakra / Mayur Kit / May 2026

File	Type	Purpose
KarmaKshetra_DK_051626_v31.html	HTML	The working Chakra™ V31 prototype. This is what gets converted.
MayurKit_052626.docx	DOCX	This document — the complete handoff brief. Ver 1.
Chakra_BrandGuidelines_051526.docx	DOCX	Brand guidelines, colors, fonts, locked terminology.
Chakra_SystemArchitecture_051626.docx	DOCX	Full system architecture and locked bucket/phase names.
Chakra_TransferFile_V31_051626.md	MD	Session transfer file with all V31 context for Claude Code.
Chakra_BrandDelta_051626.docx	DOCX	Brand updates and color system changes since Ver 0.
Chakra_BuildNotes_051626.docx	DOCX	Version history V27–V31, pending items, design decisions.
Chakra_PhilosophyLog_051626.docx	DOCX	The why behind every design decision. Required reading before coding.

⚠ Mayur must read *Chakra_PhilosophyLog* and *Chakra_SystemArchitecture* before writing a single line of code. The naming and architecture are IP — nothing changes without DK's approval.

13. Non-Negotiable Rules for Mayur

Trademarked names	Karya™, Dhairya™, Vishram™, Manan™, Manthan™, Tyaga™, Prarabdha™, Karma Kshetra™, Spandan™, Vivek™, Sthapana™, Kriya™, Mukti™, Chakra™, Sutra™ — never changed, abbreviated, or paraphrased. Ever.
Brand colors	Use only the hex codes in Section 5. No substitutions.
No 'Created by Claude'	This phrase must never appear in any file, document, or UI element.
No black backgrounds	#000000 is banned as any primary background or dominant surface.
API keys	Never hardcoded. Always in .env. Always in .gitignore. Always in Render environment variables.
Versioning	Files named as: Name_MMDDYY.ext or Name_MMDDYY_vNN.html. Version numbers never reset.
Go-live dates	Always confirmed with DK before any production deployment. No exceptions.
DNS / domain changes	Always confirmed with DK before touching GoDaddy settings.
Account naming	All platform logins use the Aux prefix as per Section 4.
Philosophy first	If any technical decision conflicts with the Gita-based naming or philosophy, stop and consult DK before proceeding.

Jai Shri Krishna 🙏

Dedicated to Lord Krishna by Dheeraj Kohli (DK)

Mayur Kit · Ver 1 · May 26, 2026 · Confidential
House of DK · Aux Services LLC · Austin, Texas